aresys

Advancing remote sensing

Sentinel-1 Level 1 Noise Processor

Denoising vectors update



Ref: ARE-030845
Issue: 1.4
Date: 06/11/2026

Document Information

Title	**Sentinel-1 Level 1 Noise Processor**
Date	06/11/2026
Aresys Reference	ARE-030845
SAR MPC Nomenclature	DI-MPC-E25
SAR MPC Reference	MPC-0680
Issue	1.4
Pages	34
Classification	Public

Recipients	**Company**
Muriel Pinheiro	ESA
Guillaume Hajduch	CLS

Prepared by	**Company**
Marco Abatangelo	Aresys
Beatrice Mai	Aresys

Checked by	**Company**
Beatrice Mai	Aresys

Approved by	**Company**
Andrea Recchia	Aresys

Change history

Date	**Issue**	**Change**	**Author**
02/04/2026	1.0	First issue of the technical note	M. Abatangelo, B. Mai
26/05/2026	1.1	Release of version 1.0.1: introduced I/Q Bias Correction for L0N noise power extraction, improved sub-swath merging strategy for GRD products	M. Abatangelo, B. Mai
22/07/2026	1.2	Release of version 1.0.2: removed processor's dependency on the statically linked C++ binary and private wheels	M. Abatangelo, B. Mai
06/11/2026	1.3	Release of version 1.1.0: changed algorithm for noise pulse discarding, azimuth FM rate selection per block for GRD products	M. Abatangelo, B. Mai

Date	Issue	Change	Author
06/11/2026	1.4	Release of version 1.2.0: added validity check on PG products and added support for SM and WV products	M. Abatangelo, B. Mai

Table of contents

1. Introduction **5**
1.1. Purpose ... 5
1.2. Acronyms ... 5
1.3. Applicable documents ... 6
1.4. Reference documents ... 6
2. Overview **7**
2.1. Denoising vector model ... 7
2.2. Noise power from noise pulses ... 7
2.3. Rank echoes ... 8
3. L1N processor **9**
3.1. Overview ... 9
3.2. Usage ... 10
3.3. Architecture and functionality ... 10
3.4. Critical points ... 12
3.4.1. Noise normalization factor ... 12
3.4.2. Noise power extraction ... 13
3.4.3. Discard of noise pulses ... 16
3.4.4. Subswath merging ... 18
3.5. Proposal for additional annotations ... 19
4. Interfaces **20**
4.1. Inputs ... 20
4.2. Outputs ... 20
4.3. Exit codes ... 21
5. User Manual **22**
5.1. Package description ... 22
5.2. Installation ... 22
5.3. Processor usage ... 22
5.3.1. Working directory ... 22
5.3.2. Job Order ... 23
5.3.3. Running the processor ... 24
6. Validation results **25**
6.1. One day analysis ... 25
6.2. Cross-type dataset ... 26
6.2.1. SLC data ... 28
6.2.2. GRD data ... 29

1. Introduction

1.1. Purpose

The present document describes the Sentinel-1 L1N processor tool, which was developed in the framework of the Evolution #25 within the SAR MPC contract.

The Sentinel-1 L1N processor has the scope of computing updated denoising vectors, for each Sentinel-1 Level 1 Annotation/Standard (and optional Level 0 Noise) product given as input via Job Order. The updated denoising vectors are stored in updated annotation files, which are structurally analogous to the noise XML files included in the `annotation/calibration/` folder of the S1 SAFE products.

This work has the purpose to allow for retro-calibration of the denoising vectors, in particular:

- to generate more accurate denoising vectors for S1 products generated before 26/06/2018 (IPF 291 deployment activating processing of L0N with rank echoes);
- to fix denoising vector bugs from IPF versions before 3.7.1, released in October 2023.

1.2. Acronyms

Acronym	Definition
ATBD	Algorithm Theoretical Basis Document
BAQ	Block Adaptive Quantization
BIPR	Background Intellectual Property Rights
CDSE	Copernicus Data Space Ecosystem
DEM	Digital Elevation Model
DGM	Detected Ground projected and Multi-looked
E2E	End-to-End
ECSS	European Cooperation for Space Standardization
ESA	European Space Agency
FDBAQ	Flexible Dynamic BAQ
GMF	Geo-Magnetic Field
GPP	Ground Processor Prototype
GRD	Ground Range Detected
HRWS	High Resolution Wide Swath
KO	Kick Off
IERS	International Earth Rotation and Reference Systems Service
IPF	Instrument Processing Facility
IRF	Impulse Response Function
ISP	Instrument Source Packets
L1A	Sentinel-1 Level 1 Annotation
L0N	Sentinel-1 Level 0 Noise
MAE	Mean Absolute Error
MPC	Mission Performance Cluster
NESZ	Noise Equivalent Sigma-Zero
OSV	Orbit State Vectors
PRF	Pulse Repetition Frequency

RCS	RADAR Cross-Section
RD	Reference Document
RFI	Radio Frequency Interferences
RFP	Request For Proposal
SAR	Synthetic Aperture Radar
SLC	Single Look Complex
SoW	Statement of Work
SRD	System Requirements Document
SW	Software
SWL	Sampling Window Length
SWST	Sampling Window Start Time
TBD	To Be Defined
TBC	To Be Confirmed
TDS	Test Data Set
TT	Task Table
WP	Work Package
XML	eXtensible Markup Language

1.3. Applicable documents

[AD1] SAR-MPC-0629, E25: *L0N generation tool and noise vector reconstruction, Proposition of evolution*
[AD2] GMES Generic PDGS-IPF Interface Specification

1.4. Reference documents

[RD1] A. Recchia, B. Mai, L. Fioretti, R. Piantanida, M. Steinisch, N. Franceschi, G. Hajduch, P. Vincent, M. Pinheiro, N. Miranda et al., *Generating Accurate De-Noising Vectors for Sentinel-1: 10 Years of Continuous Improvements*, Remote Sens. 2025, 17, 3474
[RD2] A. Recchia, D. Giudici, R. Piantanida, N. Franceschi, A. Monti-Guarnieri and N. Miranda, *On the Effective Usage of Sentinel-1 Noise Pulses for Denoising and RFI Identification*, EUSAR 2018; 12th European Conference on Synthetic Aperture Radar, Aachen, Germany, 2018
[RD3] MPC-0392, *Thermal Denoising of Products Generated by the S-1 IPF*, issue 1.1, November 2017, on-line document
[RD4] *Sentinel-1 Product Specification*, issue 3/14/1, October 2023, on-line document
[RD5] MPC-0307, *Sentinel-1 Level 1 Detailed Algorithm Definition*, issue 2/5, November 2022, on-line document

2. Overview

2.1. Denoising vector model

The thermal noise affects the quality of SAR images from both a radiometric and an interferometric point of view. The problem of estimating the noise level from SAR data is discussed in details in [RD1].

Summarizing, the SAR processor performs range and azimuth compression of the acquired raw data and implements a set of corrections aimed at compensating for the data distortions introduced by the instrument (e.g., the antenna patterns) and by the acquisition geometry (e.g., the spread losses). These corrections, applied by the S-1 IPF during L1 product generation, affect both signal and noise (see [RD1]).

The denoising procedure consists of subtracting the expected level of noise to obtain an unbiased estimate of the signal component. The model of the noise level computed by the processor is reported in the following equation:

$$n_{\mathrm{vect}}(i,j) = P_{\mathrm{n}}(i) \cdot \frac{G_{\mathrm{pg}}(i) \cdot G_{\mathrm{ds}}(i)}{G_{\mathrm{eap}}(i,j) \cdot G_{\mathrm{rsl}}(j)} \cdot \sqrt{k_{\mathrm{proc}}} \cdot \sqrt{k_{\mathrm{noise}}} \cdot \sqrt{k_{\mathrm{norm}}} \tag{1}$$

where:

- $P_{\mathrm{n}}(i)$ is the **noise power** measured from the noise pulses and rank echoes (when available);
- $G_{\mathrm{pg}}(i)$ is the **internal calibration gain correction**;
- $G_{\mathrm{ds}}(i)$ is the **de-scalloping function**;
- $G_{\mathrm{eap}}(i,j)$ is the **elevation antenna pattern gain**;
- $G_{\mathrm{rsl}}(j)$ is the **spreading loss correction**;
- k_{proc} is the **applied processing gain**;
- k_{noise} is the **noise calibration factor**, noise calibration factor that is needed to align the denoising vectors generated by IPF to the actual noise level measured in the data;
- k_{norm} is the **processor normalization factor**.

The Sentinel-1 IPF generates the noise file metadata included in Level-1 products, containing the estimated noise maps.

2.2. Noise power from noise pulses

Sentinel-1 A and B acquire dedicated noise pulses at the beginning (pre-amble) and at the end (post-amble) of each data take to estimate the noise power used to generate denoising vectors. Since the start of the mission, the estimated noise power has shown a bimodal behavior related to the area within the antenna footprint during the acquisition of the noise pulses. This effect arises from the difference in Earth emissivity between land and sea, which produces a noise level variation of approximately 1 dB between the two scenes. As a result, relying solely on the noise pulses collected in the pre- and post-amble sections of the data takes does not allow an accurate estimation of the noise level across the entire acquisition, leading to poorly calibrated denoising vectors.

To address this issue, a new denoising strategy for TopSAR products was introduced. To better track the noise power evolution within the data takes, since June 2018, the rank echoes (*i.e.* the first echoes of each burst that are signal-free and can hence be used as noise proxy) are used to generate the noise vectors in the L1 products.

Moreover, Sentinel-1 C and D instruments have been slightly evolved to improve the electronic design. The evolution allowed for a reduction in the number of internal calibration pulses from 5 to 3. In turn, the usage of less calibration pulses enabled the acquisition of noise pulses during the data-takes. Additionally, noise pulses before and after each imagette in Wave mode have been included to properly track the noise evolution within the long wave acquisitions.

2.3. Rank echoes

For Sentinel-1 A/B, the use of rank echoes significantly improved the accuracy of the noise estimation compared to the traditional approach based only on preamble and postamble noise pulses. When the latter method is used, inaccuracies in the noise estimation can produce visible discontinuities at the boundaries between adjacent subswaths. In contrast, when the noise power is estimated using rank echoes, these discontinuities are effectively mitigated because the noise level is characterized more consistently across the entire swath. This leads to a more reliable calibration of the denoising vectors and a more uniform radiometric appearance of the processed products.

For Sentinel-1 C/D, after the introduction of the additional noise pulses in the data takes, the usage of rank echoes is no longer strictly necessary. Nevertheless, the rank echoes are still available, and an assessment of their usage is on-going. The assessment is aimed at identifying the possible quality improvement in the L1 noise vectors if rank echoes are used. Finally, it is worth mentioning a second reason why usage of rank echoes could be beneficial for S1C/D. The L0N products are also used by the L1 IPF for a pre-screening step to identify RFI contamination. If RFI contamination is detected in L0N products, the RFI mitigation step is activated for the corresponding L0S product. Currently, for S-1A/B the RFI pre-screening is performed with a temporal rate lower than 1 second. For S-1C/D L0N products without rank echoes there can be gaps of 3 seconds in the pre-screening step that could results in missed RFI contamination.

3. L1N processor

3.1. Overview

The Sentinel-1 L1N processor has the scope of generating updated denoising vectors, for each S-1 L1A/S (and optional L0N) product given as input via Job Order. The updated denoising vectors are stored in updated annotation files, which are structurally analogous to the noise XML files included in the `annotation/calibration/` folder of the S1 SAFE products. For a detailed description of the structure of the noise annotation files for S1 SAFE products see [RD4].

In particular the goal is to reconstruct and compute more accurate denoising vectors (w.r.t the one annotated by IPF), starting from the information stored and available in the Level-1 annotations.

The processor reconstructs the denoising vectors annotated by the IPF using the formula reported in Equation 1. The information needed are derived, when available, from different sections of the L1 annotation XML files.

In particular:

- $P_{\mathrm{n}}(i)$ is the **noise power correction factor**, derived from the noise list annotated in the annotation XML of the SAFE product. Alternatively, it can be extracted from the L0N product, if provided as input (further details in Section 3.4.2).
- $G_{\mathrm{pg}}(i)$ is the **PG correction factor**, derived from replica list annotated in the annotation XML of the SAFE product, corrected for PG_{ref}, interpolated at the center of the burst. Only the replica records flagged as valid (both `relativePgProductValidFlag` and `absolutePgProductValidFlag` set to `true`) are used, and each discarded record is reported with a warning;
- k_{noise} is the **noise calibration factor**, read from the calibration auxiliary file (AUX_CAL);
- k_{proc} is the **processing gain**, read from L1 Processor Parameters auxiliary files (AUX_PP1), corrected for the resampling factor for all subswaths after the first one;
- $G_{\mathrm{eap}}(i, j)$ is the **elevation antenna pattern gain**, read from antenna pattern list annotated in the annotation XML of the SAFE product, and re-interpolated over the denoising vectors axis;
- $G_{\mathrm{ds}}(i)$ is the **azimuth elementary antenna pattern gain**, annotated in the noise vectors (no modification needed, and only applicable to TopSAR data);
- $G_{\mathrm{rsl}}(j)$ is the **range spreading loss** derived from slant range times, elevated to the power of 4, *i.e.*

$$G_{\mathrm{rsl}}(j) = \left(\frac{R_{\mathrm{ref}}}{t_{\mathrm{range}} \cdot \frac{c}{2}} \right)^4 \tag{2}$$

  where $c = 3 \cdot 10^8$ [m/s] denotes the speed of light and $R_{\mathrm{ref}} = 8 \cdot 10^5$ [m];

- k_{norm} is the **processor normalization factor**, computed by IPF to account for noise scaling, based on performed processing (whitening, filtering and multi-looking). The normalization factor is the product of an azimuth and a range contribution:

$$k_{\mathrm{norm}} = k_{\mathrm{norm,az}} \cdot k_{\mathrm{norm,rg}}, \qquad k_{\mathrm{norm,az}} = \frac{1}{L} \sum_{l=1}^{L} \frac{G_l}{W_l} \tag{3}$$

  where l denotes the look number and L the number of looks, *i.e.* the azimuth contribution is the average value of the ratio between the noise processing gain of the look G_l and its window normalization factor W_l, while the range contribution $k_{\mathrm{norm,rg}}$ accounts for the range processing bandwidth. We refer to Section 3.4.1 for further mathematical details.

Some of these information are annotated straightforward to find within the SAFE product, other are harder to reconstruct. This topic will be further discussed in Section 3.4.

3.2. Usage

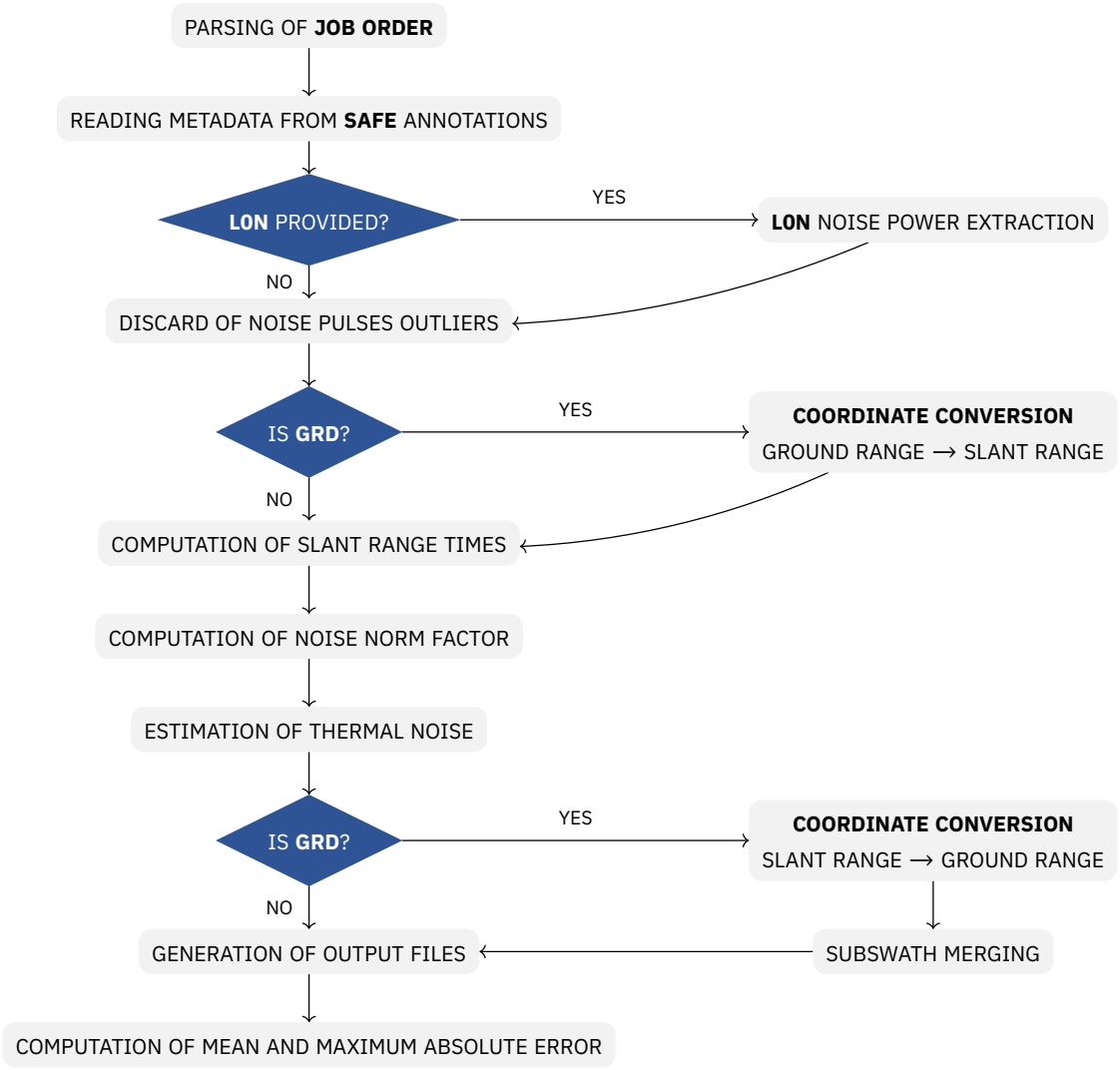
The processor has been developed to generate Level 1 noise annotations for S1A, S1B, S1C, and S1D TopSAR data-takes, including acquisitions prior to June 2018 (before rank echoes introduction for S1A and S1B).

When used in combination with the L0N processor, it enables the regeneration of higher-quality denoising vectors for L1 SAFE products produced before 2018, addressing known limitations in the original processing chain. Furthermore, the processor provides a systematic framework for the retro-calibration of existing denoising vectors, allowing previously generated products to be reprocessed with improved methodologies. This capability enhances the consistency, reliability, and overall radiometric quality of historical datasets, making them more suitable for long-term analyses and cross-sensor comparisons.

3.3. Architecture and functionality

This section explains all the steps that the candidate processor makes to generate denoising vectors starting from a given Sentinel-1 (all units) L1A/S product(s). Refer to [RD1] for more specific theoretical details.

The processing pipeline implemented within the processor is summarized in the flow chart reported below.



The following points provide a more detailed explanation for each step of the processing chain.

1. **Parsing of Job Order**
   The processor reads every file path given as input within the Job Order and checks whether they are reachable from the current working directory. The expected input files are:
   - L1A/S product filepath(s) – all Sentinel-1 units are supported, *i.e.* S1A/B/C/D;
   - AUX_INS filepath;
   - AUX_CAL filepath;
   - AUX_PP1 filepath;
   - L0N product filepath(s) – **optional**.

   Some additional checks are performed, including:
   - verifying that the AUX files provided by the user are the same as those referenced in the manifest of the considered L1A input product, in order to minimize the computed error. If different AUX files are detected, or an AUX file is not referenced in the manifest, a warning is issued, but processing continues;
   - automatically pairing of L1A/S and L0N SAFE products, when multiple L1A/S input products belonging to different data-takes are provided together with the corresponding L0N products. Products are paired when mission, acquisition mode, absolute orbit number and datatake ID in their names match; if several L0N products match the same L1A/S product, the first one is used and a warning is issued;
   - ensuring that the specified output folder for the results is different from the input folder to avoid pollution of input product folders.
2. **Reading metadata from SAFE annotations**
   Input files are parsed and stored in ad-hoc classes for quicker and easier access. This includes auxiliary files.
3. **L0N noise power extraction (optional)**
   If no L0N product is provided as input, the noise power is derived from the noise list annotated in the annotation XML of the L1A/S SAFE product, following the steps listed at point 3 of Section 3.4.2. Alternatively, whenever a L0N product is provided as input to the processor, the noise power is extracted from the Level 0 noise annotations and filtered. The noise extraction and filtering procedures are explained more in detail in Section 3.4.2.
4. **Discard of noise pulses**
   Whether noise power is extracted from L1A annotations or extracted from L0N product(s) (when provided as input), noise measurements outside an interval around the expected noise level read from AUX_INS are discarded as outliers. This is explained more in detail in Section 3.4.3.
5. **Coordinate conversion**
   If the input is a GRD product, the `coordinateConversion` section in its annotations is used to extract a 9th-degree polynomial that converts ground range data into slant range. This allows the SLC algorithms to be reused without introducing additional pipeline differences between SLC and GRD products.
6. **Computation of slant range times**
   For each burst in each subswath, slant range times are computed for processing purposes, together with range distances and range spread loss.
7. **Computation of noise norm factor**
   The IPF processor computes internally a multiplicative constant for each burst to correct noise estimations. This involves complex computations like IPF processor's shrink factor and Hamming windowing for each look of the acquisition. This is explained more in detail in Section 3.4.1.
8. **Estimation of thermal noise**
   The estimated thermal noise of the burst is calculated as

   ```
   simulated_noise = np.sin(incidence_angles) * noise_power * G_tot *
   noise_norm_factor
   ```

   where `G_tot` is the global radiometric correction factor, which is the combination of all the radiometric corrections applied by the processor:

```
G_tot = pg**2 * k_N * k_proc / (elevation_pattern * range_spread_losses)
```

following Equation 1. In the GRD case, noise values are converted back from slant range to ground range.

9. **Subswath merging**
   The de-noising process depends on sub-swath dependent parameters: if the input is a GRD product, the processor needs to separate the sub-swaths merged inside the same annotation file, compute thermal noise values on each sub-swath and merge them together to output a single, cross-sub-swath file. A detailed step-by-step breakdown of this procedure is available in Section 3.4.4.
10. **Generation of output files**
    Noise range vectors are written on an XML file. Noise azimuth vectors are not modified. The denoise files are named after their respective noise files, with the initial prefix "*de*" being the only difference in the filename. The output is stored in the output folder specified by the user in the Job Order, which cannot be the same as the input product(s).
11. **Computation of mean absolute error**
    Thanks to the previous step, the denoise and noise files are parsed (and interpolated in the GRD case) to compute the mean absolute error (MAE) and provide an estimate of the quality of the denoising procedure.

3.4. Critical points

In this section, the main difficulties encountered during the development phase of the processor are presented, and possible solutions are proposed.

3.4.1. Noise normalization factor

One of the most challenging task was to reverse-engineer how the IPF computes its processing normalization factor, since it takes into account the whitening process, the filter used and the multi-looking process. This quantity is not only product-dependent, but **burst-dependent**. The only way to retrieve its correct value was to manually re-implement all the processing steps used within the IPF to estimate it, including Hamming windowing, angle computation, antenna pattern interpolation and correction, and window normalization.

As explained in Section 3.1, we compute the noise normalization factor as the product of an azimuth and a range contribution, $k_{\mathrm{norm}} = k_{\mathrm{norm,az}} \cdot k_{\mathrm{norm,rg}}$.

3.4.1.1. Azimuth contribution

The azimuth contribution is the average over the L looks of the noise normalization factor of each look. Namely, for the l-th look, we have

$$k_{\mathrm{norm},l} = \frac{G_l}{W_l} \tag{4}$$

where G_l is the noise processing gain and W_l is the window normalization factor of the look.

The looks are extracted on an azimuth FFT grid of N_{FFT} samples, *i.e.* the smallest valid FFT length not lower than the number of lines per burst, with frequency resolution $\Delta f = f_{\mathrm{az}}/N_{\mathrm{FFT}}$, where f_{az} is the azimuth sampling frequency. The quantities involved are derived from the `azimuthProcessing` section of the product annotation:

- $N_{\mathrm{bw}} = \mathrm{round}(B_{\mathrm{az}}/\Delta f)$ is the processing bandwidth in samples, with B_{az} the annotated `processingBandwidth`;
- $N_w = \mathrm{round}(B_{\mathrm{look}}/\Delta f)$ is the length of each look, with B_{look} the annotated `lookBandwidth`;
- $w(\cdot)$ is the Hamming window of length N_w, whose coefficient is the annotated `windowCoefficient`;
- N_{ov} is the look overlap in samples, computed as in IPF:

$$N_{\mathrm{ov}} = \left\lceil \frac{L \cdot N_w - N_{\mathrm{bw}}}{L-1} \right\rceil \tag{5}$$

for $L > 1$, and $N_{\mathrm{ov}} = 0$ for single-look processing;
- k_l is the start index of look l on the FFT grid:

$$k_l = \left[\left[\frac{1 - N_{\mathrm{ext}}}{2}\right] + (l-1)\cdot(N_w - N_{\mathrm{ov}})\right] \bmod N_{\mathrm{FFT}}, \qquad N_{\mathrm{ext}} = N_w + (L-1)\cdot(N_w - N_{\mathrm{ov}}) \tag{6}$$

- $S(k)$ is the azimuth antenna pattern (AAP) read from AUX_CAL, in linear amplitude, interpolated in dB on the FFT grid. The angular step corresponding to one frequency bin is $\frac{\lambda}{2v}\cdot\Delta f\cdot(1 - k_{\mathrm{DC}}/k_{\mathrm{FM}})$, where v is the satellite velocity and $1 - k_{\mathrm{DC}}/k_{\mathrm{FM}}$ is the TopSAR shrink factor. The Doppler centroid rate k_{DC} is derived from the azimuth steering rate of the subswath read from AUX_INS (`radarParams/azimuthSteeringRate`), and the azimuth FM rate k_{FM} from the `azimuthFmRateList` of the product annotation, both evaluated at mid-range. For SLC products the FM rate record of the burst is used; for GRD products, whose FM rate records are not aligned with the blocks of each subswath, the record closest in time to the block is used.

The expressions of G_l and W_l depend on whether the AAP correction has been applied by the IPF, as reported by the `antennaAzimuthPatternApplied` flag in the product annotation. When the AAP correction is applied, the spectrum is divided by the AAP, which also boosts the noise at the spectrum edges:

$$G_l = \frac{1}{N_{\mathrm{bw}}}\sum_{k=0}^{N_w-1}\left[\frac{w(k)}{S(k_l+k)}\right]^2, \qquad W_l = \frac{\sum_{k=0}^{N_w-1} w(k)^2}{\sum_{k\in\mathcal{B}} S(k)^2} \tag{7}$$

where $\mathcal{B}$ denotes the N_{bw} samples of the processing bandwidth, centered on the zero frequency. When the AAP correction is not applied, the AAP is not divided out and remains in the spectrum of each look:

$$G_l = \frac{1}{N_{\mathrm{bw}}}\sum_{k=0}^{N_w-1} w(k)^2, \qquad W_l = \frac{\sum_{k=0}^{N_w-1}\left[S(k_l+k)\cdot w(k)\right]^2}{\sum_{k\in\mathcal{B}} S(k)^2} \tag{8}$$

All indices are taken modulo N_{FFT}.

3.4.1.2. Range contribution

In range the spectrum is flat, so the noise normalization only depends on the rounding of the processing bandwidth to an integer number of samples:

$$k_{\mathrm{norm,rg}} = \frac{N_{\mathrm{bw,rg}}}{\mathrm{round}(N_{\mathrm{bw,rg}})}, \qquad N_{\mathrm{bw,rg}} = \frac{B_{\mathrm{rg}}}{\Delta} f_{\mathrm{rg}} \tag{9}$$

where B_{rg} is the annotated range processing bandwidth,

$$\Delta f_{\mathrm{rg}} = f_s / N_{\mathrm{FFT,rg}} \tag{10}$$

with f_s being the range sampling frequency after decimation and $N_{\mathrm{FFT,rg}}$ being the smallest valid FFT length not lower than the number of samples per burst. This contribution is close to 1, but not negligible when targeting small reconstruction errors.

With the support of multi-look processing and of products without AAP correction, no empirical correction of the noise normalization factor is needed anymore (previous releases applied one to the EW1 subswath of EW GRDM products).

Within the same subswath, the value of this normalization factor varies by no more than 0.0001% between consecutive bursts (this is due to small differences in consecutive FM rate values, incidence angle variance, floating-point imprecision, and so on), while differences across subswaths can reach up to 26%. Reporting this value directly in the Level 1 annotations would eliminate the need for these re-computations altogether, reducing the remaining uncertainty to the negligible burst-to-burst variation within each subswath.

3.4.2. Noise power extraction

The noise power extraction from L0N products is performed as follows.

1. **Raw packets are decoded** into I/Q samples. Preamble, rank echo, and noise pulse measurements are compressed with plain Block Adaptive Quantization (BAQ, 3/4/5-bit modes) as described in [RD5], section

9.1, and are decoded into the raw quantizer codes accordingly – valid as long as each block's Threshold Index (THIDX) stays below the ICD reconstruction threshold ([RD5], subsection 9.1.2.2), which always holds for the low-dynamic-range noise measurements used here and is verified at runtime.

2. **I/Q Bias Correction is applied** as discussed in [RD5], section 9.2.1. Moreover, since for Sentinel-1 data the instrument's receive module performs the demodulation in the digital domain, the I/Q gain imbalance and I/Q non-orthogonality corrections are not necessary. This means that the aforementioned correction is equivalent to enforce zero mean on both components of the signal.
3. **Mean noise power is computed** by reading and averaging separately each noise sequence from:
   - preamble measurements;
   - rank echoes measurements (extracting one mean noise power value for each rank echo sequence);
   - noise pulses, only present for S1C/D (extracting one mean noise power value for each noise sequence);
   - postamble measurements.
4. **Some rank echoes are discarded** as they cannot all be used for noise estimation. Specifically, some echoes are affected by the stabilization pulses in the preamble, as well as by the TxCal/TxHCalIso pulses, as described in [RD5], subsection 4.2.1.9. In particular:
   - spikes observed in the rank echo measurements of the first burst of the data-take are removed for S1A/B (in IW1 and EW1 subswaths) as they are biased by previous instrument calibration and stabilization pulses;
   - the last rank echo in a EW1 sequence for S1A/B is removed since nadir returns might be present;
   - rank echoes are filtered in an alternating pattern for S1A/B IW products, since odd-numbered measurements are biased by previous instrument calibration pulses;
   - rank echoes measurements for EW2 are replaced with those derived from EW3, for all Sentinel-1 units. If EW3 is not available, the EW2 measurements are kept; if the two sequences have different lengths, only the overlapping measurements are replaced. A warning is issued in both cases.

As discussed later in Section 6 of the document, when analyzing the results using external L0N products to derive noise power values, instead of using the noise power annotated by the IPF, the S1 L1N processor does not exhibit a reduction in precision, as shown in Figure 6. An example of noise power extraction for a S1A EW product is provided in Figure 1, where the difference between the noise power level extracted from the L1A product and L0N product is lower than 0.05 dB.

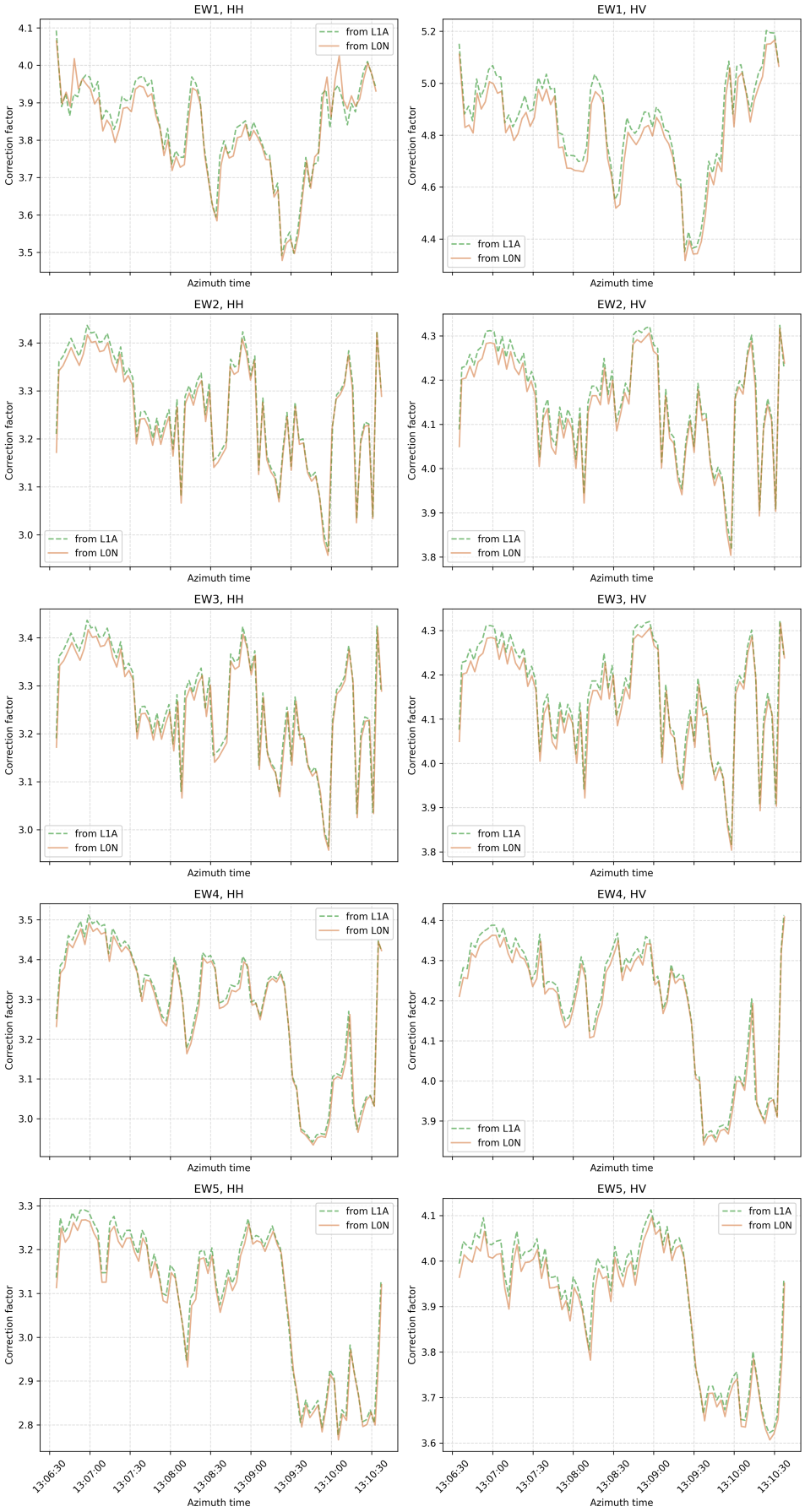


Figure 1: comparison of results of noise power extraction performed on a S1A EW product. Noise power extraction from L1A product in dashed **green** vs from L0N product in **orange**.

3.4.3. Discard of noise pulses

3.4.3.1. IPF's current approach

In the IPF processor, before using the noise power estimates to generate a noise vector, a validation step is performed: the differences between consecutive noise estimates are compared against a threshold and the noise measure is discarded in case the threshold is exceeded, as described in [RD5], sections 4.2.1.9 and 4.2.2.

Being based only on the evaluation of power variation between consecutive noise measurements, this approach may encounter some issues: in cases when multiple consecutive noise measurements show high power values, these will not be discarded.

3.4.3.2. L1N implementation

In S1 L1N processor a different approach is proposed: instead of relying on the statistics of the noise sequence itself, each noise measurement is compared with the expected noise level of the instrument. For each channel (subswath and polarization), the AUX_INS file provides the expected noise power P_{exp} (`noise` field of the internal calibration parameters). A noise measurement y_i is retained only if it falls within an interval around this value:

$$P_{\text{exp}} \cdot 10^{-\Delta_{\text{low}}/10} \leq y_i \leq P_{\text{exp}} \cdot 10^{\Delta_{\text{up}}/10} \tag{11}$$

with $\Delta_{\text{low}} = 3$ dB and $\Delta_{\text{up}} = 1$ dB. Given a sequence of observations $Y = \{y_1, y_2, ..., y_n\}$ in linear scale, the cleaned dataset is therefore:

$$Y_{\text{clean}} = \left\{ y_i \in Y : P_{\text{exp}} \cdot 10^{-\Delta_{\text{low}}/10} \leq y_i \leq P_{\text{exp}} \cdot 10^{\Delta_{\text{up}}/10} \right\} \tag{12}$$

The interval is asymmetric: measurements exceeding the expected level by more than 1 dB, as in case of RFI contamination, are discarded, while the tolerance toward lower values is wider. Since the reference is external to the noise sequence, the criterion does not depend on the number of contaminated measurements: even when several consecutive noise pulses are affected by RFI, they are still identified and discarded. Moreover, no minimum number of observations is required. The filter is applied both to the noise power annotated in the L1A/S product and to the one extracted from L0N products, and each discarded noise pulse is reported in the processor log. If all the noise measurements of a channel are discarded, the nominal expected noise power P_{exp} read from AUX_INS is used as a constant noise power for the whole channel, and a warning is issued.

The outlier detection is implemented as follows:

```
def remove_outliers(azimuth_times, noise_pulses, expected_noise_value):

  azimuth_times = np.array(azimuth_times)
  noise_pulses = np.array(noise_pulses, dtype=float)
  min_value = expected_noise_value * db_to_linear(-NOISE_LOWER_BOUND)
  max_value = expected_noise_value * db_to_linear(NOISE_UPPER_BOUND)

  # Detect outliers
  mask = (noise_pulses < min_value) | (noise_pulses > max_value)
  indices = np.where(mask)[0]

  # Collect outlier info (already sorted by original index)
  outlier_indices = [[idx, azimuth_times[idx], noise_pulses[idx]] for idx in indices]

  # Remove outliers
  azimuth_times_clean = azimuth_times[~mask]
  noise_pulses_clean = noise_pulses[~mask]

  return azimuth_times_clean, noise_pulses_clean, outlier_indices
```

where `NOISE_LOWER_BOUND` and `NOISE_UPPER_BOUND` are the bounds Δ_{low} and Δ_{up} in dB. They are set by default to 3.0 and 1.0 respectively, but they can be customized in future releases or be set directly in the Job Order.

3.4.3.3. Example of application

These two different approaches for discarding outliers from the noise power estimates may yield to different results in case of RFI contamination. An example of this can emerge considering the data-take

`S1C_IW_GRDH_1ADV_20260421T040125_20260421T040150_007309_00ED1F_E017.SAFE`

In Figure 2 it is reported a visualization from the CDSE browser of a portion of this data-take, comprehensive of the denoising step. This means that the noise maps estimated from the IPF have already been removed from the data. Two things can be observed:

- RFI contamination is clearly present both in IW2 and IW3 subswaths;
- for IW3 (subswath on the left in Figure 2) a wrong level of noise has been estimated and removed from the data, causing a saturation in the color scale visualization.

This happened because, for this particular example, the RFI contaminated more than one consecutive noise pulse for IW3, resulting in a failure of outlier identification by the IPF.

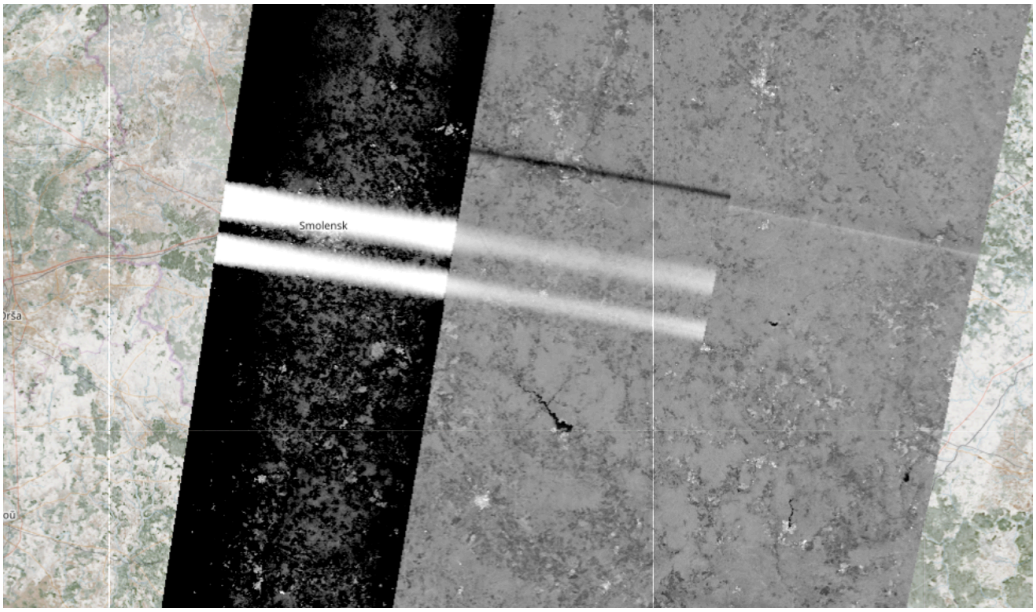
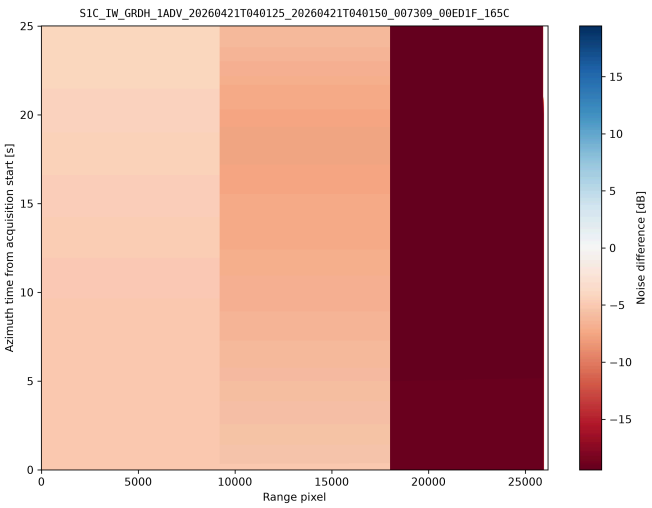


Figure 2: IW VV polarization, decibel γ^0 layer, just outside Moscow on April 21st 2026, from CDSE browser.

Regarding the noise maps estimations, comparing nominal noise annotations and the output from the S1 L1N processor, the latter computes a more consistent and continuous thermal noise level across the whole acquisition. On the right, the difference of the annotated and reconstructed noise maps is shown: a pronounced discrepancy is observable within the IW3 subswath.



Looking at noise profiles for a fixed azimuth time, this is even clearer. A general over-estimation by the IPF of the noise level is present for all three subswaths of this product, but in particular for IW3 a completely wrong level is computed, due to the RFI contamination within the noise pulses.

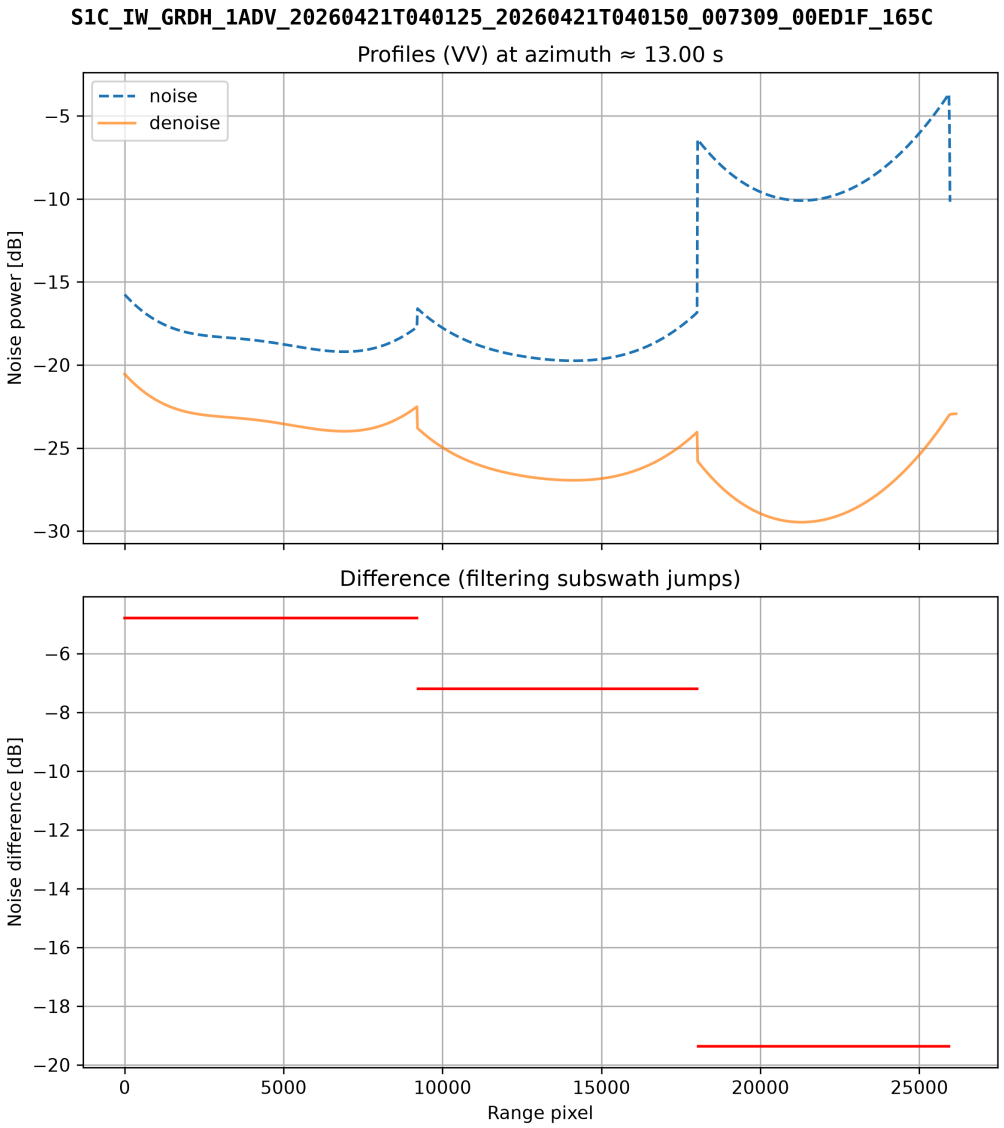


Figure 3: comparison between annotated and reconstructed noise profiles, VV polarization.

3.4.4. Subswath merging

The denoising process relies on a set of parameters that are specific to each subswath and therefore cannot be applied uniformly across the entire image at once. When the input is a GRD product, the processor is required to first identify and separate the individual subswaths that have been merged together within a single annotation file. It then computes the thermal noise values independently for each subswath, before finally recombining all of them into a single, continuous cross-subswath output file. Once all the necessary noise values have been computed across the raster, the merging procedure is carried out through the following steps.

1. The noise vectors belonging to each subswath are extended via interpolation so that they cover the full azimuth extent of the scene.
2. For each azimuth time step, the individual subswaths are concatenated along the range direction, with special care taken to properly handle any regions where neighboring subswaths overlap.
3. The resulting merged noise values are then consolidated into a single noise vector for each azimuth line.

The outcome of this procedure is the final cross-subswath denoise annotation, which provides a seamless and consistent noise characterization across the entire image.

3.5. Proposal for additional annotations

In this section we summarize a list of additional annotations that would be helpful to include within the official SAFE product in order to facilitate the noise vectors reconstruction process. Those are not currently included in the IPF outputs, but they might be considered for inclusion in upcoming versions.

- A list containing the **noise normalization factor** k_{norm}.
  This quantity is product, channel (subswath and polarization) and burst dependent. Since the variation within the same channel is negligible, a single annotation for each channel can be proposed. The reason of why this would be useful is discussed in Section 3.4.1.
- Associating **quality flags to noise pulses** to provide an indication of which noise sequences to use, and which needs to be discarded, for the noise power extraction when using L0N products.
  This information is channel (subswath and polarization) dependent. The reason of why this would be useful is discussed in Section 3.4.2. Moreover, it would be useful to add the information on which noise sequences are actually discarded by the IPF processor. The L1N processor discards the noise measurements outside an interval around the expected noise level read from AUX_INS, as discussed in Section 3.4.3.

4. Interfaces

The S1 L1N processor generates, inside the working directory, new Level 1 denoising annotations starting from L1A/S SAFE products. Indeed, since the processor only takes as input the metadata from the product provided, L1S products can be passed as inputs as well: the binary files will be ignored.

4.1. Inputs

To correctly run the S1 L1N processor expects the working directory to be configured as follows:

1. **Job Order:** an XML file providing information about the location of all input and output files/folders. See Section 5.3.2 for more detailed information about the Job Order file.
2. **L1A/S folder(s):** target SAFE products for the generation of denoising files. Currently, the processor can correctly handle Level 1 (A/S) SLC and GRD TopSAR acquisition from missions S1A, S1B, S1C, and S1D.
3. **Auxiliary Data files:** some parameters for the original IPF processing are mission-dependent and written in the relative AUX_PP1, AUX_CAL and AUX_INS folders. Those need to be provided as input from the user. In particular, AUX_INS also provides the expected noise power used to discard noise pulse outliers (Section 3.4.3) and the azimuth steering rate of each subswath used for the noise normalization factor (Section 3.4.1).
4. **L0N folder(s) – optional:** if a relevant Level 0 Noise SAFE product is provided, the processor will re-compute the noise power correction, instead of reading it directly from the Level 1 annotations of the target product(s).

4.2. Outputs

S1 L1N creates an output folder named after the target input L1A/S product, containing the denoise files. The output files are stored in a separate folder (w.r.t. the input SAFE products) without modifying the input L1A/S SAFE products. The generated denoise files are divided by subswath and polarization in case of a SLC input products, or by polarization only for the GRD case. An example is reported below.

```
output/
├── S1A_EW_GRDM_1ADV_20250915T223810_20250915T223914_061004_0799D6_EA11/
│   ├── denoise-s1a-ew-grd-vh-20250915t223810-20250915t223914-061004-0799d6-002.xml
│   └── denoise-s1a-ew-grd-vv-20250915t223810-20250915t223914-061004-0799d6-001.xml
└── S1A_EW_SLC__1ADH_20250906T230229_20250906T230314_060873_079499_988E/
    ├── denoise-s1a-ew1-slc-hh-20250906t230229-20250906t230312-060873-079499-001.xml
    ├── denoise-s1a-ew1-slc-hv-20250906t230229-20250906t230312-060873-079499-006.xml
    ├── denoise-s1a-ew2-slc-hh-20250906t230230-20250906t230313-060873-079499-002.xml
    ├── denoise-s1a-ew2-slc-hv-20250906t230230-20250906t230313-060873-079499-007.xml
    ├── denoise-s1a-ew3-slc-hh-20250906t230230-20250906t230313-060873-079499-003.xml
    ├── denoise-s1a-ew3-slc-hv-20250906t230230-20250906t230313-060873-079499-008.xml
    ├── denoise-s1a-ew4-slc-hh-20250906t230231-20250906t230314-060873-079499-004.xml
    ├── denoise-s1a-ew4-slc-hv-20250906t230231-20250906t230314-060873-079499-009.xml
    ├── denoise-s1a-ew5-slc-hh-20250906t230231-20250906t230314-060873-079499-005.xml
    └── denoise-s1a-ew5-slc-hv-20250906t230231-20250906t230314-060873-079499-010.xml
```

The denoise files mirror the content of the noise annotations inside the input SAFE product(s), whose structure is presented in [RD4], section 6.3.3. An example of denoise file content is shown below.

```
<?xml version='1.0' encoding='UTF-8'?>
<noise>
  <adsHeader>
    <missionId>S1B</missionId>
```

```
    <productType>SLC</productType>
    <polarisation>VH</polarisation>
    <mode>IW</mode>
    <swath>IW2</swath>
    <startTime>2021-12-15T17:19:30.517919</startTime>
    <stopTime>2021-12-15T17:19:55.698484</stopTime>
    <absoluteOrbitNumber>30039</absoluteOrbitNumber>
    <missionDataTakeId>235062</missionDataTakeId>
    <imageNumber>002</imageNumber>
  </adsHeader>
  <noiseRangeVectorList count="9">
    <noiseRangeVector>
      <azimuthTime>2021-12-15T17:19:30.517920</azimuthTime>
      <line>0</line>
      <pixel count="669">0 40 80 120 160 ... 26692</pixel>
      <noiseRangeLut count="669">3.800881e+02 ... 4.273933e+02</noiseRangeLut>
    </noiseRangeVector>
    ...
    <noiseRangeVector>
      <azimuthTime>2021-12-15T17:19:52.594595</azimuthTime>
      <line>12088</line>
      <pixel count="669">0 40 80 120 160 ... 26692</pixel>
      <noiseRangeLut count="669">3.853264e+02 ... 4.199797e+02</noiseRangeLut>
    </noiseRangeVector>
  </noiseRangeVectorList>
  <noiseAzimuthVectorList count="1">
    <noiseAzimuthVector>
      <swath>IW2</swath>
      <firstAzimuthLine>0</firstAzimuthLine>
      <firstRangeSample>0</firstRangeSample>
      <lastAzimuthLine>13598</lastAzimuthLine>
      <lastRangeSample>26692</lastRangeSample>
      <line count="1368">0 10 20 30 40 ... 13598</line>
      <noiseAzimuthLut count="1368">1.111833e+00 ... 1.112657e+00</noiseAzimuthLut>
    </noiseAzimuthVector>
  </noiseAzimuthVectorList>
</noise>
```

4.3. Exit codes

S1 L1N processor terminates with one of the exit codes summarized in the following table.

Value	Mnemonic	Description
0	OK	The processor execution was successfully completed.
128	FAILURE	Error exit code, returned also for invalid command line arguments and unexpected errors. The processing is considered failed.

5. User Manual

5.1. Package description

The S1 L1N processor is delivered as a **Python package**, exposing the `denoise` CLI and accepting the Job Order interface described in this document.

The package is published on PyPI as `s1-denoise` and is also distributed as a single, platform-independent wheel:

`s1_denoise-M.m.p-py3-none-any.whl`

where `M`, `m` and `p` indicate respectively the major, minor and patch version. All dependencies are automatically resolved from PyPI by `pip` at install time.

5.2. Installation

The package can be installed with `pip` into a Python ≥ 3.11 environment on any supported platform, either directly from PyPI:

```
pip install s1-denoise
```

optionally pinning a specific version with `pip install s1-denoise==M.m.p`, or from the wheel file:

```
pip install s1_denoise-M.m.p-py3-none-any.whl
```

Both options install the `denoise` command into the active Python environment.

5.3. Processor usage

5.3.1. Working directory

An example of a configured working directory is reported below.

```
working_directory/
├── aux-files/
│   ├── S1D_AUX_CAL_V20260106T000000_G20260120T110643.SAFE
│   ├── S1D_AUX_INS_V20251110T000000_G20251217T143507.SAFE
│   └── S1D_AUX_PP1_V20251110T000000_G20251105T150133.SAFE
├── input/
│   ├── S1D_EW_GRDM_1ADH_20260202T084121_20260202T084145_001305_00154E_08FF.SAFE
│   ├── S1D_EW_RAW__0NDH_20260202T084121_20260202T084144_001305_00154E_F66D.SAFE
│   └── S1D_IW_SLC__1ADV_20260205T171338_20260205T171405_001354_00174E_B095.SAFE
├── JobOrder.s1d-ew-grd-l0n.xml
├── JobOrder.s1d-iw-slc.xml
└── output/
    ├── S1D_EW_GRDM_1ADH_20260202T084121_20260202T084145_001305_00154E_08FF
    │   ├── denoise-s1d-ew-grd-hh-20260202t084121-20260202t084145-001305-00154e-001.xml
    │   └── denoise-s1d-ew-grd-hv-20260202t084121-20260202t084145-001305-00154e-002.xml
    └── S1D_IW_SLC__1ADV_20260205T171338_20260205T171405_001354_00174E_B095
        ├── denoise-s1d-iw1-slc-
vh-20260205t171339-20260205t171404-001354-00174e-001.xml
        ├── denoise-s1d-iw1-slc-
vv-20260205t171339-20260205t171404-001354-00174e-004.xml
        ├── denoise-s1d-iw2-slc-
vh-20260205t171340-20260205t171405-001354-00174e-002.xml
        ├── denoise-s1d-iw2-slc-
vv-20260205t171340-20260205t171405-001354-00174e-005.xml
```

```
        ├── denoise-s1d-iw3-slc-
vh-20260205t171338-20260205t171403-001354-00174e-003.xml
        └── denoise-s1d-iw3-slc-
vv-20260205t171338-20260205t171403-001354-00174e-006.xml
```

Input SAFE products and auxiliary files do not need to be collected in a single folder, as long as the Job Order correctly points to the target data paths.

5.3.2. Job Order

An example of a correctly filled Job Order is provided below.

```
<?xml version="1.0" encoding="UTF-8"?>
<Ipf_Job_Order>
  <Ipf_Conf>
    <Processor_Name>S1_L1N_IPF</Processor_Name>
    <Version>1.1.0</Version>
    <Stdout_Log_Level>INFO</Stdout_Log_Level>
    <Stderr_Log_Level>INFO</Stderr_Log_Level>
    <Test>false</Test>
    <Breakpoint_Enable>false</Breakpoint_Enable>
    <Processing_Station>MTI</Processing_Station>
    <List_of_Dynamic_Processing_Parameters count="1">
      <Dynamic_Processing_Parameter>
        <Name>Mission_Id</Name>
        <Value>S1C</Value>
      </Dynamic_Processing_Parameter>
    </List_of_Dynamic_Processing_Parameters>
  </Ipf_Conf>
  <List_of_Ipf_Procs count="1">
    <Ipf_Proc>
      <Task_Name>S1_L1N_IPF</Task_Name>
      <Task_Version>1.1.0</Task_Version>
      <List_of_Inputs count="7">
        <Input>
          <File_Type>IW_SLC__1A</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./input-s1c/S1C_IW_SLC__1ADV_..._0067C8_B378.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>IW_RAW__0N</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./input-s1c/S1C_IW_RAW__0NDV_..._0067C8_9086.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>IW_SLC__1A</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./input-s1c/S1C_IW_SLC__1ADH_..._006C0A_53FA.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>IW_RAW__0N</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
```

```
          <List_of_File_Names count="1">
            <File_Name>./input-s1c/S1C_IW_RAW__0NDH_..._006C0A_5DA9.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>AUX_PP1</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./S1C_AUX_PP1_V20250224T000000_G20250521T131402.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>AUX_CAL</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./S1C_AUX_CAL_V20250224T000000_G20250519T123501.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
        <Input>
          <File_Type>AUX_INS</File_Type>
          <File_Name_Type>Physical</File_Name_Type>
          <List_of_File_Names count="1">
            <File_Name>./S1C_AUX_INS_V20250224T000000_G20251015T082332.SAFE</File_Name>
          </List_of_File_Names>
        </Input>
      </List_of_Inputs>
      <List_of_Outputs count="1">
        <Output>
          <File_Type>DENOISE</File_Type>
          <File_Name_Type>Directory</File_Name_Type>
          <File_Name>./output</File_Name>
        </Output>
      </List_of_Outputs>
    </Ipf_Proc>
  </List_of_Ipf_Procs>
</Ipf_Job_Order>
```

Supported file types for input files to process are: `IW_SLC__1A`, `IW_SLC__1S`, `EW_SLC__1A`, `EW_SLC__1S`, `IW_GRDH_1A`, `IW_GRDH_1S`, `IW_GRDM_1A`, `IW_GRDM_1S`, `EW_GRDH_1A`, `EW_GRDH_1S`, `EW_GRDM_1A`, `EW_GRDM_1S`.

Supported file types for **optional** input L0N files for noise power extraction are: `EW_RAW__0N` and `IW_RAW__0N`.

`Directory` is the only type of supported output file name type. Supported file type for output is only `DENOISE`.

Setting the log level from `<Stdout_Log_Level>` and `<Stderr_Log_Level>` in the Job Order is also supported: it can be either set to `INFO` (default) or `DEBUG` (for development purposes only).

5.3.3. Running the processor

After preparing the working directory, the S1 L1N processor can be run with:

```
denoise "/path/to/job/order.xml"
```

while the installed version can be checked at any time with `denoise --version`. All available options can be listed with `denoise --help`. The processor log is printed on the console and also written to `denoise.log` in the current working directory. Subsequent runs from the same directory append to the same file.

6. Validation results

In this section the test results for the first release of the candidate processor are presented.

6.1. One day analysis

To validate the processor over a significant dataset, a whole day of acquisitions was considered for each mission, to ensure no orbit-dependent bias was introduced in the processor's algorithms. For this purposes, only SLC products were considered, with no additional L0N input. The table below reports in detail which day has been selected for the validation test and how many products have been considered, for each operating satellite (S1A/C/D). Note that during operational acquisitions EW products are significantly less than IW for a single day.

Mission ID	Day of acquisition	Number of products
S1A	03/03/2025	574 (558 IW + 16 EW)
S1C	26/02/2026	439 (432 IW + 7 EW)
S1D	17/04/2026	494 (404 IW + 90 EW)

The quality of the denoising process has been determined by computing the mean absolute error (MAE) and the maximum absolute error of the final product on the whole acquisition, expressed in dB. In Figure 4 and Figure 5 the results over the described test dataset are shown. The mean and the maximum absolute errors between annotated and reconstructed noise maps are reported for the different missions and acquisition modes. The average MAE for all analyzed cases is very small, around 0.016 dB. This indicates a very good capability of noise vectors reconstruction by the processor for SLC products.

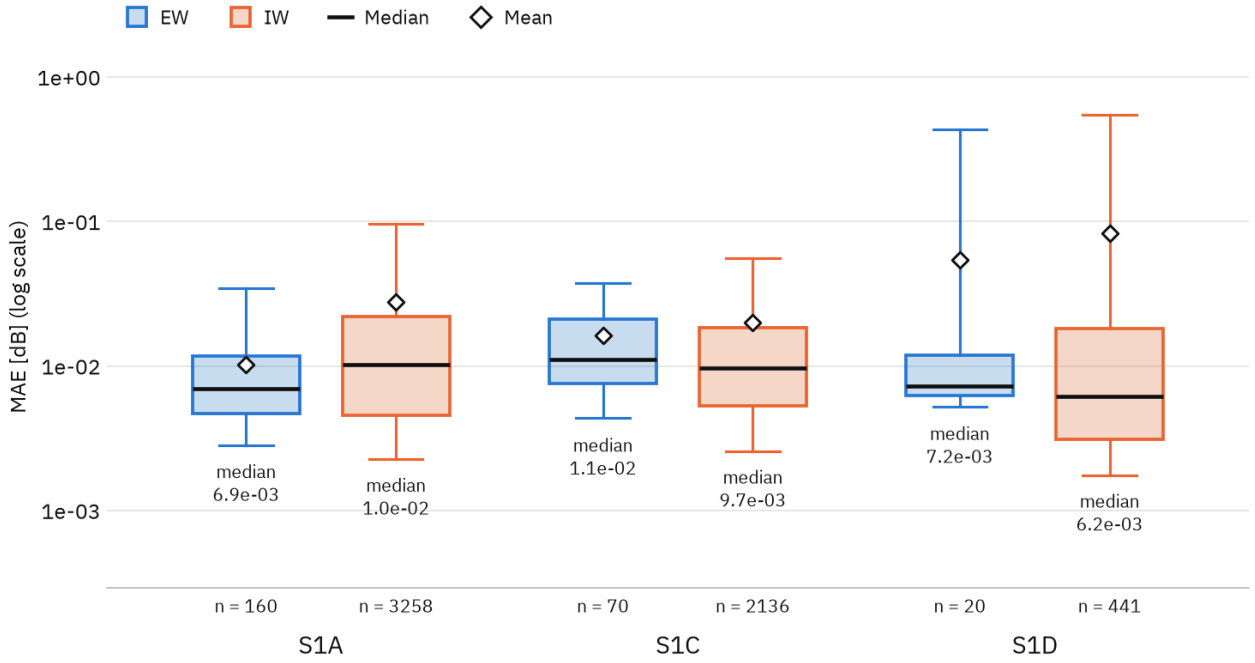


Figure 4: **mean** absolute errors between annotated and reconstructed noise maps over all products from the test dataset (box: 25th–75th percentile, whiskers: 5th–95th percentile). The numbers n at the bottom of the columns indicate the number of denoise files (i.e. subswath + polarization channels) generated and parsed for error calculation.

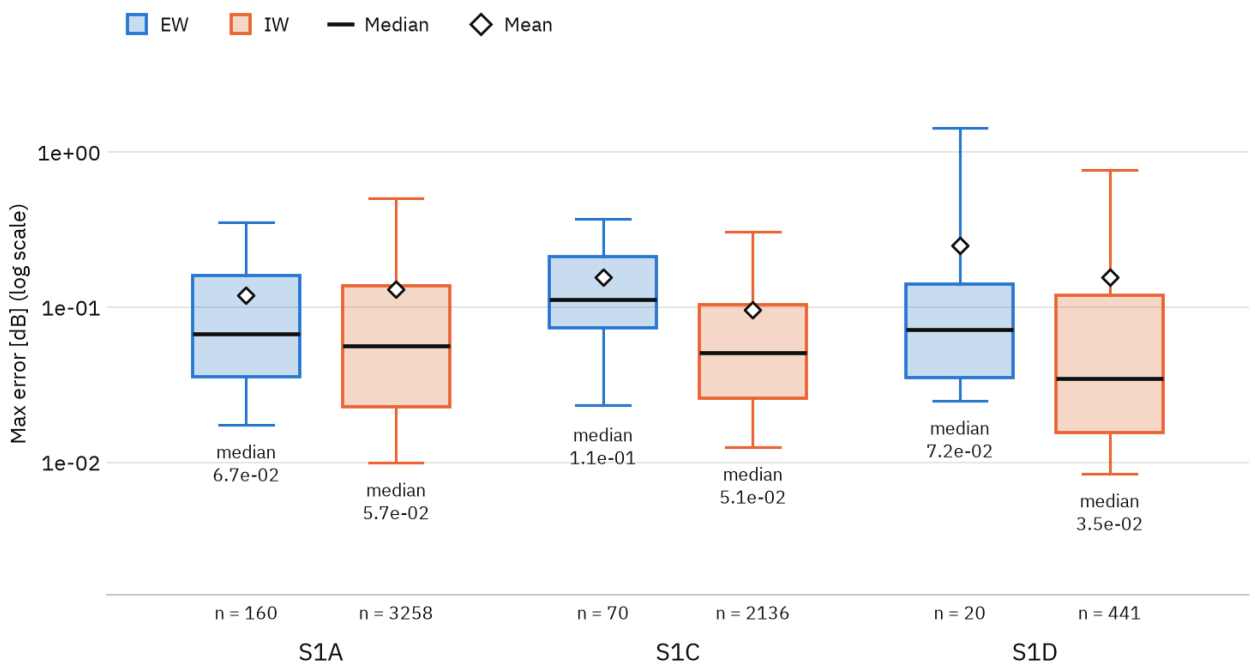


Figure 5: average **maximum** absolute errors between annotated and reconstructed noise maps over all products from the test dataset (box: 25th–75th percentile, whiskers: 5th–95th percentile). The numbers n at the bottom of the columns indicate the number of denoise files (i.e. subswath + polarization channels) generated and parsed for error calculation.

6.2. Cross-type dataset

In addition, the processor was tested on all three operating units (S1A, S1C, and S1D) using both SLC and GRD data, with and without the L0N product as an additional input. The considered test dataset is composed by the products listed in the following table:

ID	L1A product name
S1A	S1A_EW_GRDM_1ADH_20250906T230229_20250906T230314_060873_079499_B099 S1A_EW_GRDM_1ADV_20250915T223810_20250915T223914_061004_0799D6_EA11 S1A_EW_SLC__1ADH_20250906T230229_20250906T230314_060873_079499_988E S1A_EW_SLC__1ADV_20250915T223810_20250915T223916_061004_0799D6_34D2 S1A_IW_GRDH_1ADH_20250302T101348_20250302T101413_058124_072DD8_5166 S1A_IW_GRDH_1ADV_20250505T172037_20250505T172102_059062_07539E_4F63 S1A_IW_SLC__1ADH_20250302T101348_20250302T101415_058124_072DD8_FD58 S1A_IW_SLC__1ADV_20250505T171215_20250505T171243_059062_07539E_4E4F
S1C	S1C_EW_GRDM_1ADH_20260226T125724_20260226T125824_006527_00D27D_533E S1C_EW_GRDM_1ADV_20260222T225359_20260222T225441_006474_00D0B5_D099 S1C_EW_SLC__1ADH_20260226T125823_20260226T125925_006527_00D27D_ABEF S1C_EW_SLC__1ADV_20260222T225359_20260222T225441_006474_00D0B5_B47D S1C_IW_GRDH_1ADH_20260226T094036_20260226T094101_006525_00D266_D8C1 S1C_IW_GRDH_1ADV_20260226T042240_20260226T042310_006522_00D246_2B4A S1C_IW_SLC__1ADH_20260226T094035_20260226T094102_006525_00D266_0DC1 S1C_IW_SLC__1ADV_20260226T042240_20260226T042310_006522_00D246_D3DF
S1D	S1D_EW_GRDM_1ADH_20260421T193955_20260421T194100_002449_00406A_49A4 S1D_EW_GRDM_1ADV_20260421T192114_20260421T192214_002449_004066_447B S1D_EW_SLC__1ADH_20260421T194158_20260421T194301_002449_00406A_4855 S1D_EW_SLC__1ADV_20260421T192012_20260421T192115_002449_004066_F5EE S1D_IW_GRDH_1ADH_20260420T185333_20260420T185358_002434_003FF4_3355 S1D_IW_GRDH_1ADV_20260420T183231_20260420T183300_002433_003FF0_C3A0

ID	L1A product name
	S1D_IW_SLC__1ADH_20260420T185307_20260420T185334_002434_003FF4_6454 S1D_IW_SLC__1ADV_20260420T183231_20260420T183302_002433_003FF0_4C94

For each L1A product, every processing was repeated twice: one exploiting the additional L0N noise power extraction, the other reading directly the noise power from L1A annotations. The L0N products considered are listed in the following table:

ID	L0N product name
S1A	S1A_EW_RAW__0NDH_20250906T230229_20250906T230314_060873_079499_F0C7 S1A_EW_RAW__0NDV_20250915T223810_20250915T223929_061004_0799D6_4D40 S1A_IW_RAW__0NDH_20250302T101255_20250302T101509_058124_072DD8_4752 S1A_IW_RAW__0NDV_20250505T171148_20250505T172257_059062_07539E_6E38
S1C	S1C_EW_RAW__0NDH_20260226T125520_20260226T130108_006527_00D27D_75AA S1C_EW_RAW__0NDV_20260222T225400_20260222T225441_006474_00D0B5_D3A3 S1C_IW_RAW__0NDH_20260226T093917_20260226T094215_006525_00D266_7487 S1C_IW_RAW__0NDV_20260226T042241_20260226T042310_006522_00D246_2F62
S1D	S1D_EW_RAW__0NDH_20260421T193956_20260421T194428_002449_00406A_6AFC S1D_EW_RAW__0NDV_20260421T191910_20260421T192546_002449_004066_A043 S1D_IW_RAW__0NDH_20260420T185240_20260420T185554_002434_003FF4_888B S1D_IW_RAW__0NDV_20260420T183231_20260420T183355_002433_003FF0_6FD8

The quality of the denoising process has been determined by computing the mean absolute error of the final product on the whole acquisition, expressed in dB. Figure 6 reports the MAE over the described cross-type dataset, where it can be noted that the precision in the denoising procedure is not significantly biased by the use of L0N products to extract noise power values. The highest average MAE over all analyzed cases is very small, around $1.25 \cdot 10^{-1}$ dB. This indicates a very good capability of noise vectors reconstruction by the processor even for GRD products.

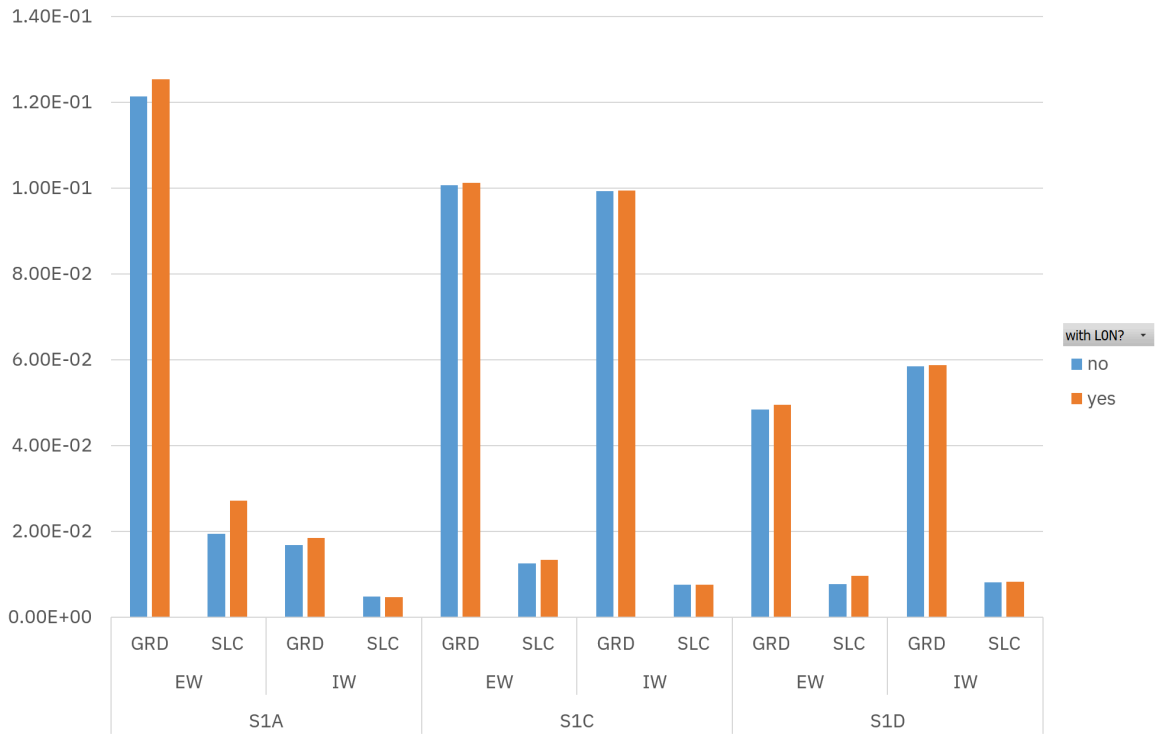


Figure 6: mean absolute errors between annotated and reconstructed noise maps on all TDS products, for all operating missions (S1A/C/D), acquisition modes (IW, EW), levels (SLC, GRD), and usage (or not) of L0N products.

6.2.1. SLC data

Regarding SLC data, the MAE obtained over the test dataset described in Section 6.2 are reported below in Figure 7. The noise maps reconstruction for SLC products is more precise: the mean absolute error is extremely small, around $1.09 \cdot 10^{-2}$ dB when L0N noise is not involved, while $1.31 \cdot 10^{-2}$ dB when re-computing the noise power (from L0N), as shown in the graph in Figure 7.

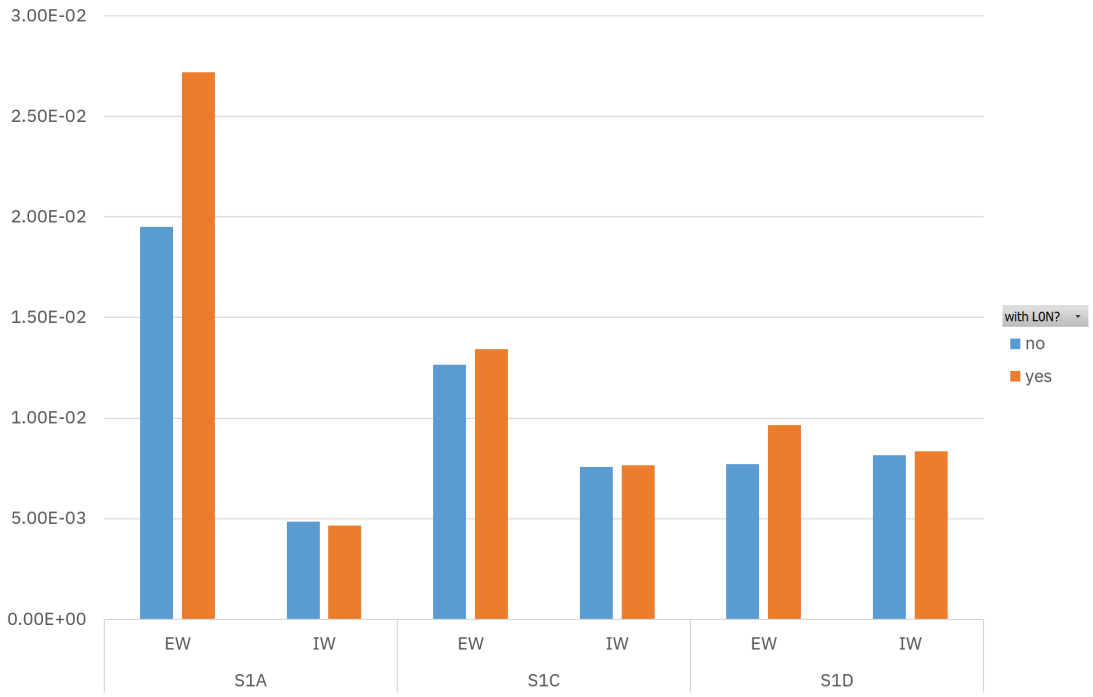


Figure 7: mean absolute errors between annotated and reconstructed noise maps on SLC products.

In Figure 8 is reported a direct comparison between the denoise files generated by S1 L1N processor and the nominal noise files, for an S1A IW SLC product.

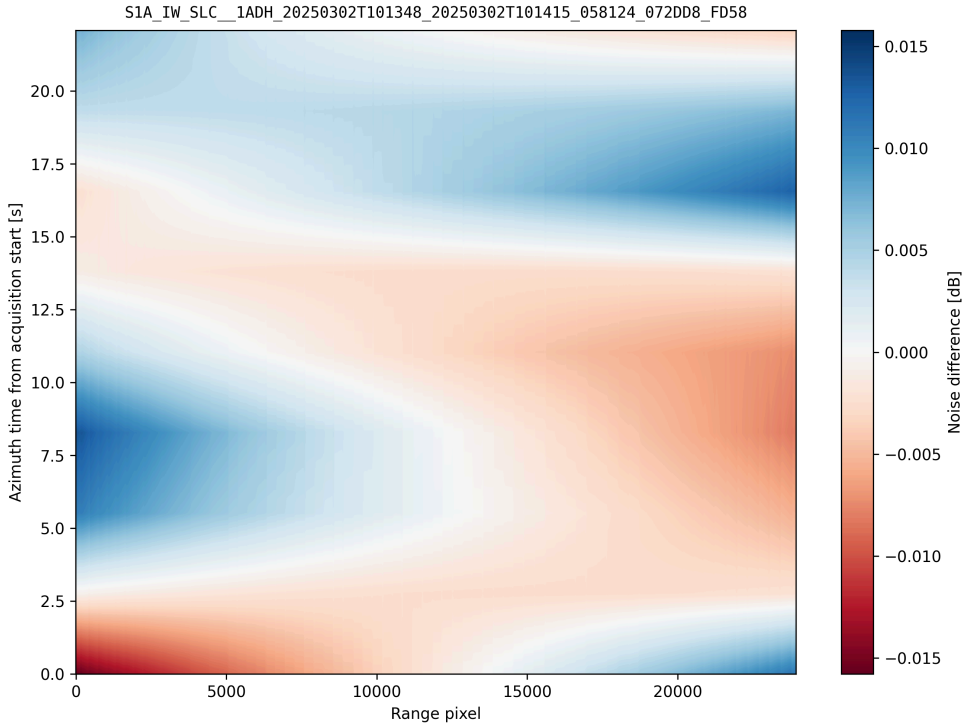


Figure 8: error between annotated and reconstructed noise maps on IW3, HH polarization, no L0N.

When comparing nominal noise profiles with reconstructed ones at a fixed azimuth time (therefore cutting Figure 8 on a fixed ordinate), some oscillations are visible. Considering, for instance, subswath EW1 and polarization VH of a S1C SLC product, looking at Figure 9, the error over the whole profile is very low, of the order of 10^{-3} dB. The red oscillations present along the profile are introduced by floating point precision and numerical errors.

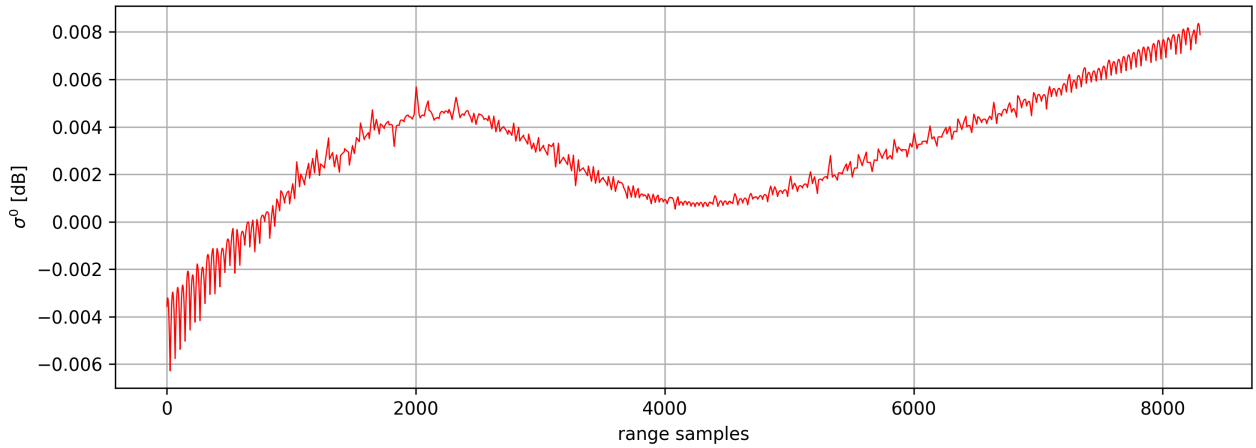


Figure 9: error between annotated and reconstructed noise profiles, for product `S1C_EW_SLC__1ADV_20250502T191252_20250502T191355_002156_00493A_0D61`, no L0N.

6.2.2. GRD data

Regarding GRD data, the MAE obtained over the test dataset described in Section 6.2 are reported below in Figure 10. In the GRD case, the error on the denoising process is slightly higher than in the SLC case: it is $7.4 \cdot 10^{-2}$ dB in standard mode and $7.5 \cdot 10^{-2}$ dB in L0N extraction mode (see Figure 10). This is in any case very small, and it is considered acceptable for the scope of this work.

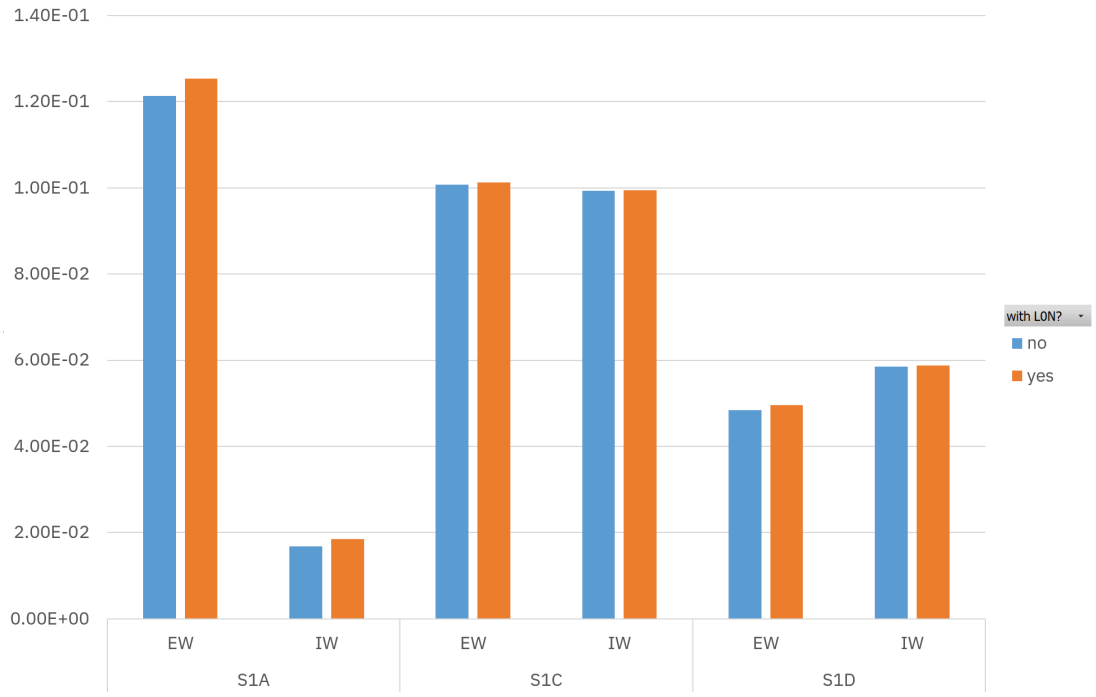


Figure 10: mean absolute errors between annotated and reconstructed noise maps on GRD products.

As anticipated, the GRD case is more delicate and requires thorough estimations to match exactly nominal noise values. One major difference from the SLC case is the ability of correctly separate, process and merge subswaths from a single annotation file. In some cases, like the GRDH products, this procedure is easier and the error stays low. An example is shown in Figure 11 for an S1C IW GRDH product.

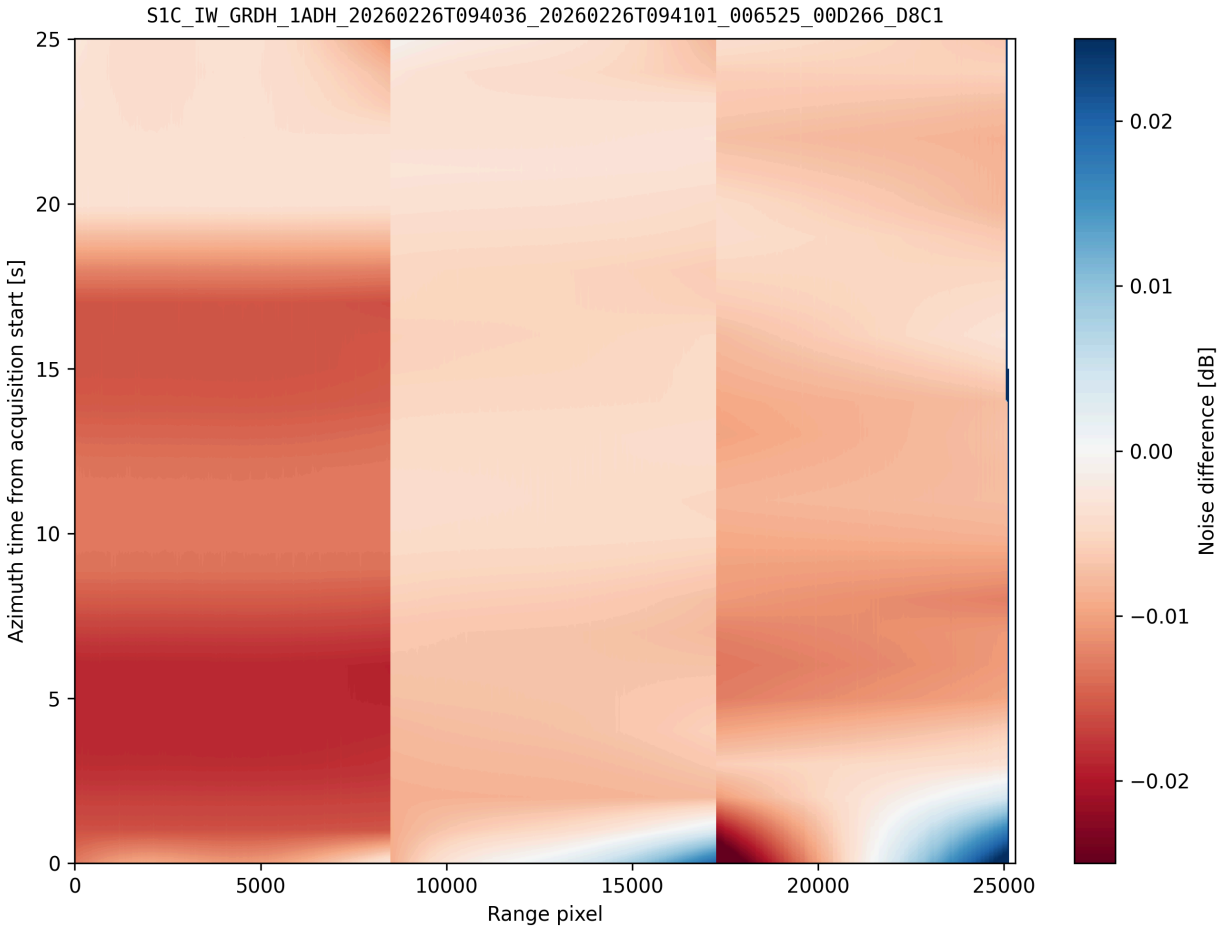


Figure 11: difference of annotated and reconstructed noise map, HV polarization.

On the other hand, GRDM products have more irregular acquisition borders, therefore they are more prone to numerical imprecision. A couple of examples over a Sentinel-1 A and C EW GRDM products are reported respectively in Figure 12 and Figure 13, where high-error areas are visible in the rightmost part of the graph (Figure 12) or at its bottom (Figure 13).

At the far-range edge of outermost subswaths, the nominal annotations set the last 1-3 noise LUT samples to zero, as visible in Figure 14 and Figure 15. This is an empirically observed behavior of the IPF output, consistent with the noise estimator lacking valid samples at the extreme antenna beam edge. Our model computes a physically meaningful value there by design, so small residual differences at boundary pixels are expected and do not indicate a processing error.

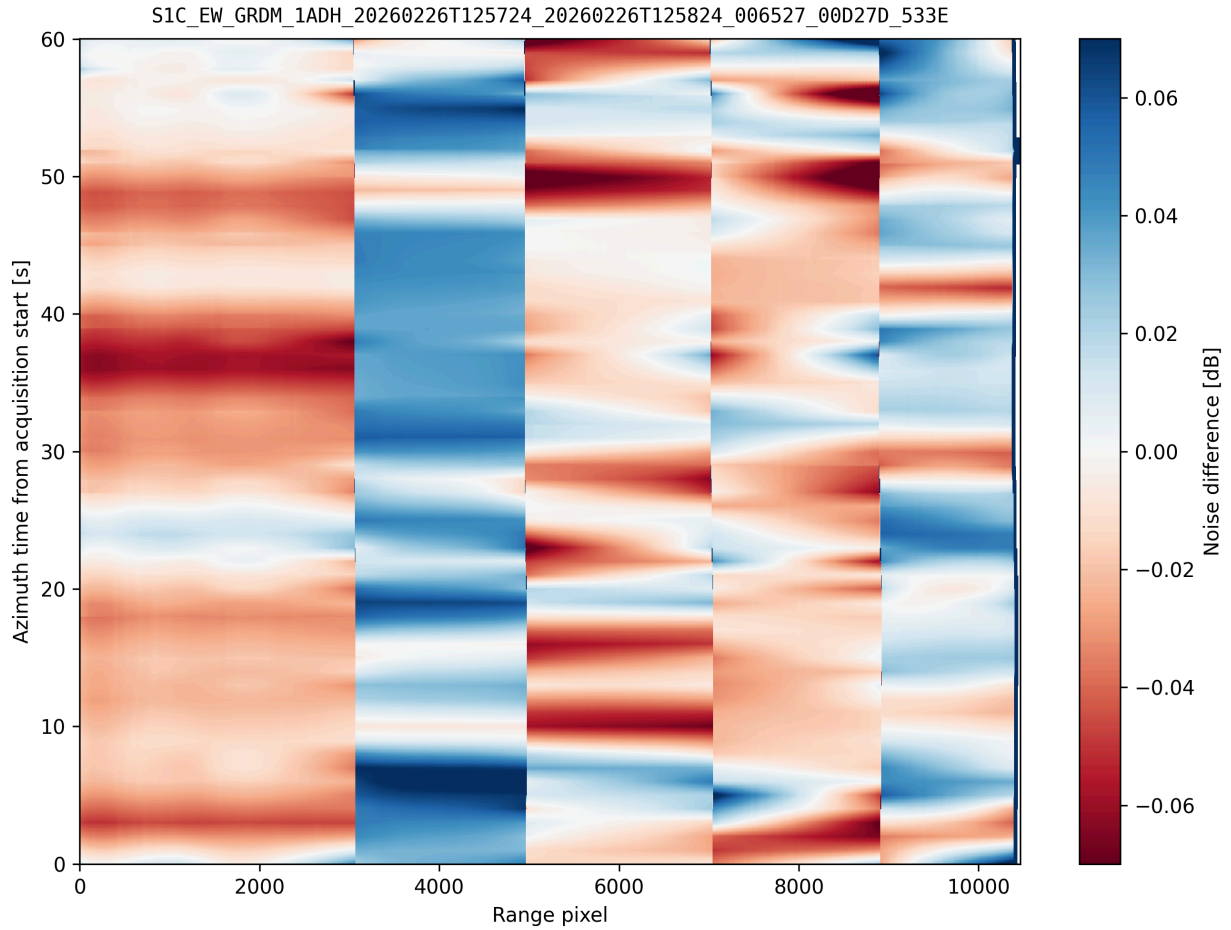


Figure 12: difference of annotated and reconstructed noise map, HV polarization.

At the start of the acquisition, narrow horizontal high-error bands may appear across all subswaths, because each subswath's first valid burst begins slightly after the product `startTime`, requiring backward extrapolation of the noise profile. IPF's empirical estimator, working directly on the measured data, may yield different values in those initial seconds. The artifact is confined to the opening seconds of the scene and does not affect overall noise model quality.

As a consequence, any interpolation-based comparison will produce an arbitrarily large dB difference in that region regardless of model accuracy. The maximum absolute error is therefore always dominated by this artifact and is not a meaningful quality indicator; the mean absolute error is the appropriate figure of merit.

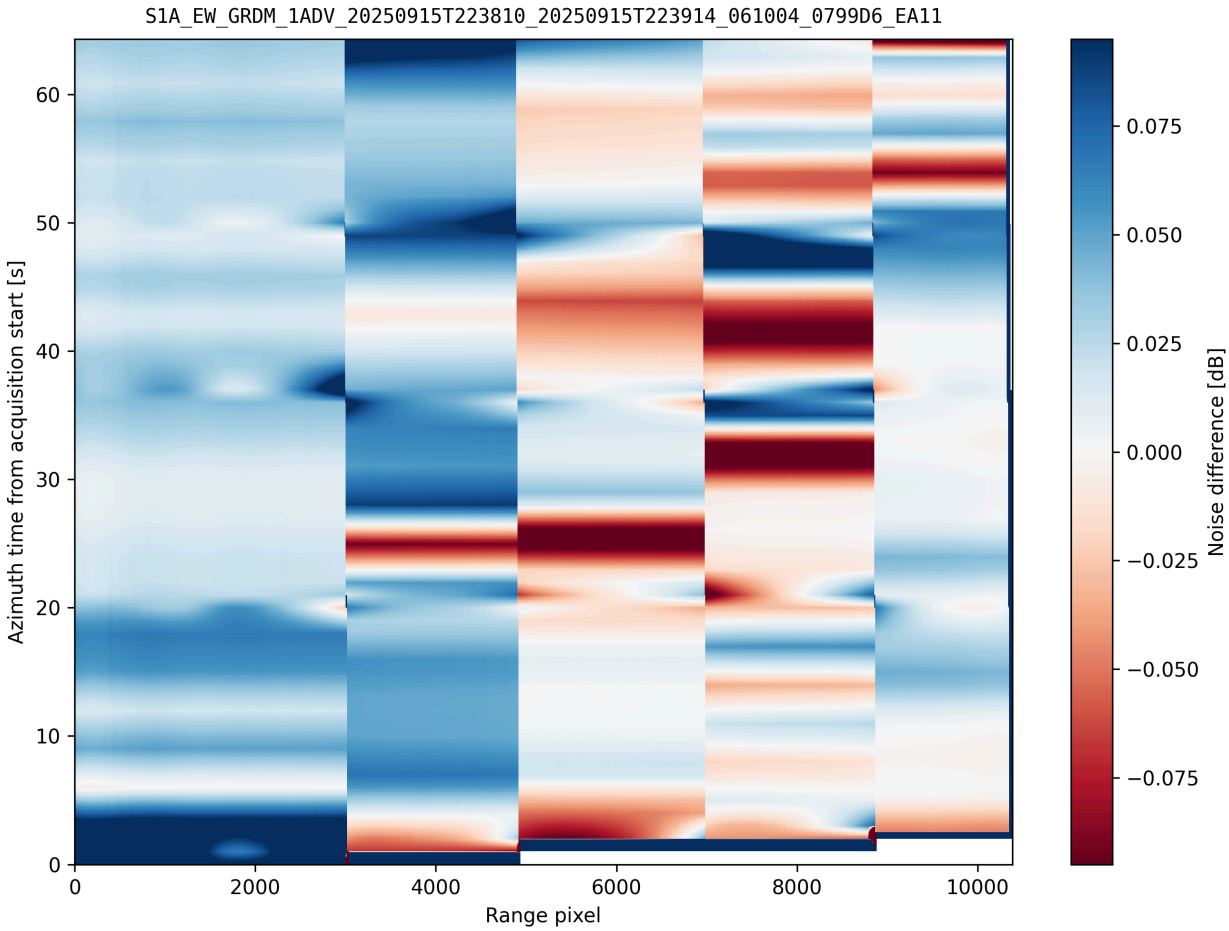


Figure 13: difference of annotated and reconstructed noise map, VH polarization.

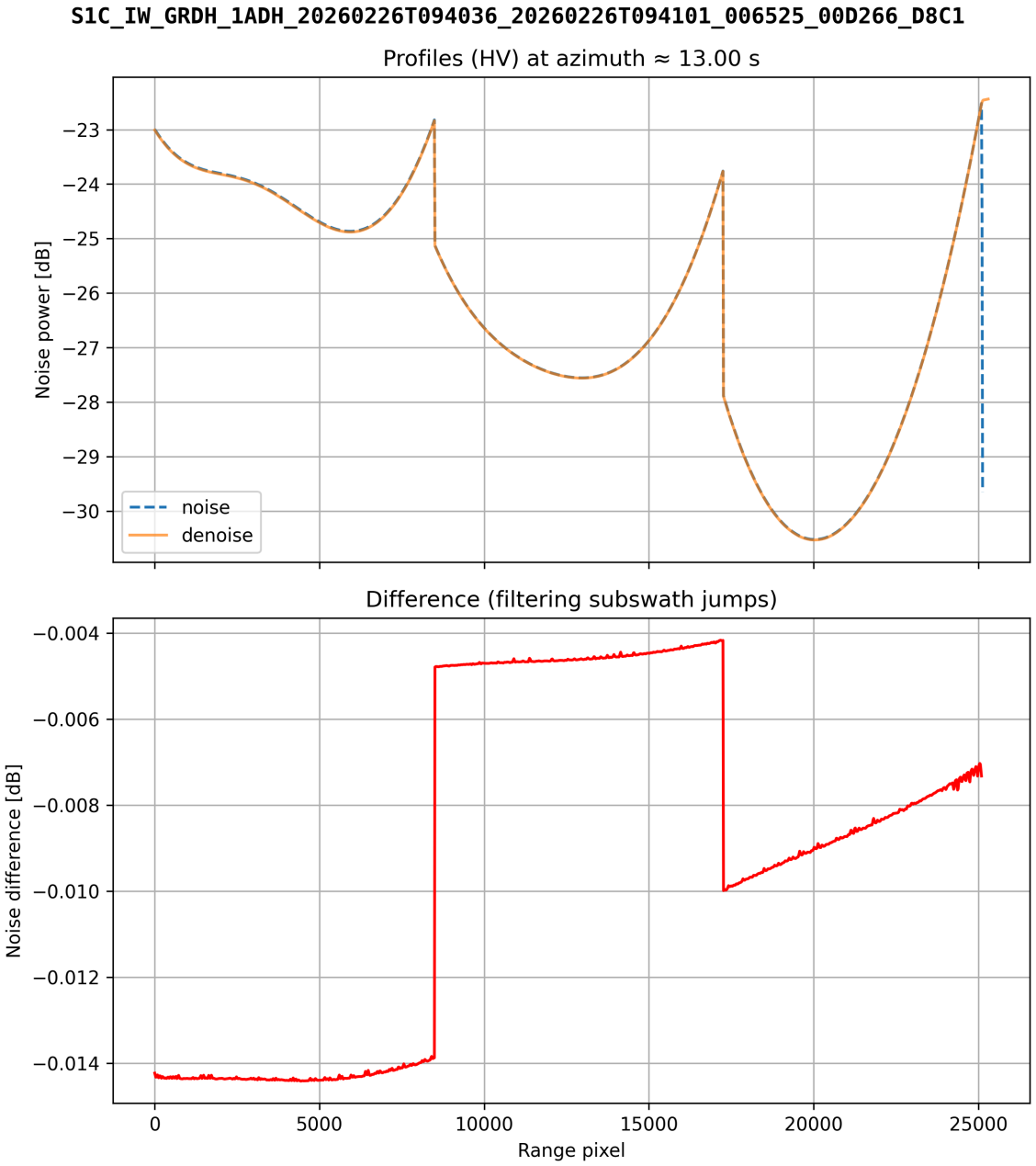


Figure 14: comparison between annotated and reconstructed noise profiles, HV polarization.

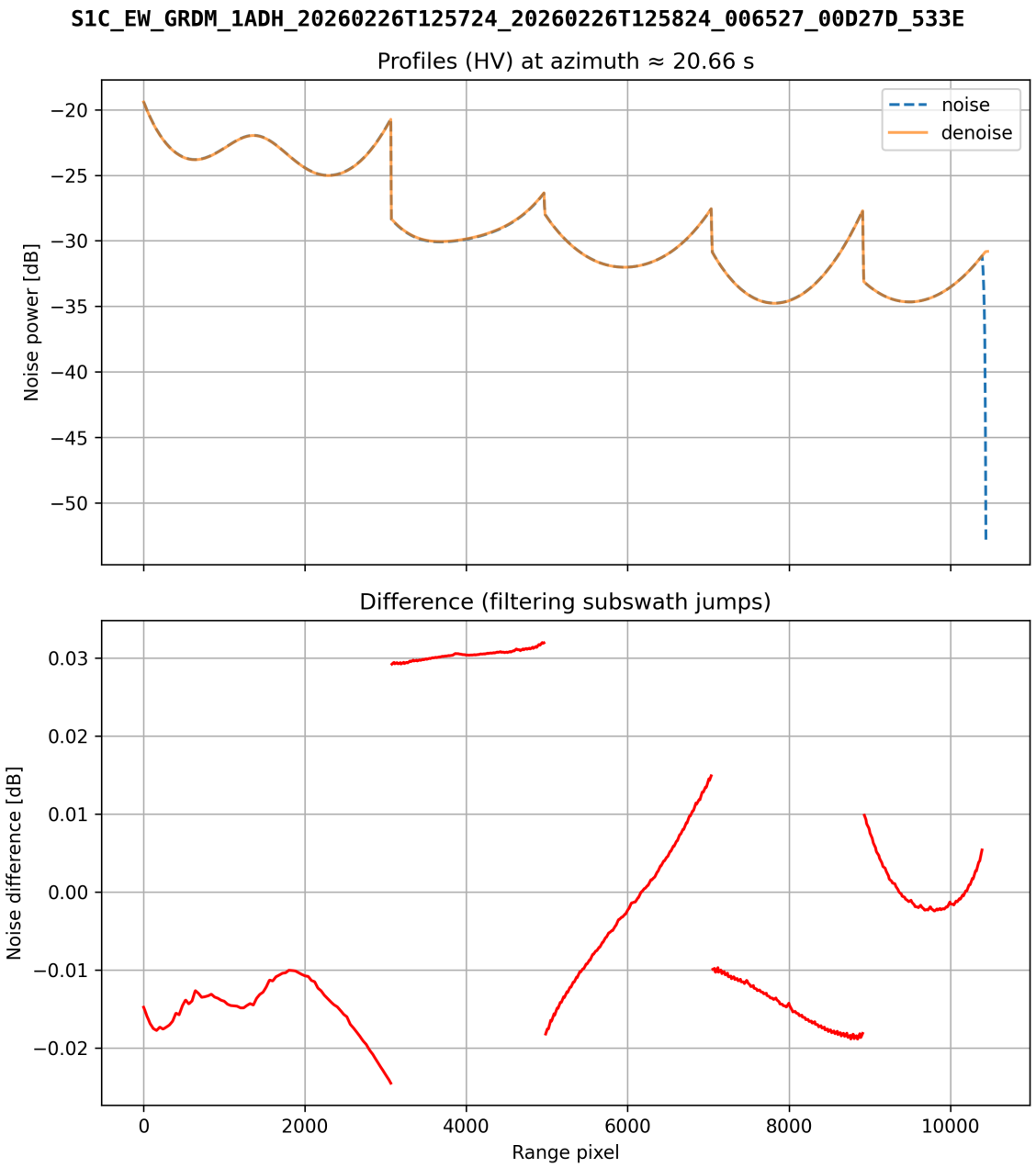


Figure 15: comparison between annotated and reconstructed noise profiles, HV polarization.